

Loan Portfolio, Delinquency & Collections Intelligence System

Rahul Meena · IIT Gandhinagar · Portfolio Project · MySQL 8.0 and Python

A lending analytics system built on 58.4 million rows of real consumer-credit data. It answers a question every lender faces and most answer by instinct: when a collections team can only call a fraction of delinquent customers each day, which ones should they call first?

1. The problem statement

1.1 The business situation

A consumer lender holds a book of active loans and credit cards. At any moment, some customers are behind on payments. A collections team can contact only a limited number of them each day, so the order of that queue determines what actually gets recovered.

The instinctive ordering is by severity: call the customers who are furthest behind first. It feels obviously correct, because the worst accounts look like the biggest problem. Most collections queues are built exactly this way.

The question this project set out to test

Is "call the worst accounts first" actually the right ordering, and if not, what is the right one, and how much does the difference matter?

1.2 Who this system is for

This is worth stating precisely, because it is easy to assume the wrong thing.

It is not	a screen where an agent looks up one customer and reads their balance and payment history. That is a customer-service tool.
It is not	a report for the customer themselves.
It is	a tool for the collections manager , who arrives on Monday with 7,211 delinquent accounts on the system, capacity for perhaps 300 calls that day, and a decision to make about where to point the team.

The output is therefore a **ranked list**, not a record. Who to call first, second, third.

1.3 The same problem, in a familiar setting

It is hospital triage

In an emergency room, patients are not treated in the order they arrived, and not by who has been waiting longest. They are treated by a combination of *how serious the condition is* and *how treatable it is*. Someone waiting six hours with a small cut goes after someone who just walked in with chest pain.

Sorting a collections queue by "most days overdue" is the same mistake as treating patients by "longest waiting". This project builds the triage rule instead: **how much money is at stake, multiplied by how likely it is to actually come back**. Both of those were measured from the data, not assumed.

1.4 The second question: does past behaviour predict future default?

A customer already holds a loan with the bank — a phone EMI, say, paid over the past year, some months on time and some late. That same customer now returns and asks for a new, larger loan.

The bank must decide whether to approve it. And the bank already holds this person's entire payment record on the existing loan.

Does how they paid the old loan tell you whether the new one will go bad?

It feels as though it must. The second half of this project tests whether it actually does, rather than assuming it.

The test is possible because the dataset contains **307,511 customers whose real outcome is already known** — whether each one did in fact run into payment difficulty on the new loan. Every claim can therefore be checked against reality instead of argued for.

1.5 Why 58 million rows produce a 7,211-row answer

The scale needs explaining, because the input and the output look mismatched.

The 58 million rows are not the answer. They are how the *rule* is worked out. To establish that accounts 1 to 30 days late recover on their own roughly 55% of the time, while accounts past 90 days recover only about 5% of the time, you have to observe millions of accounts moving month by month over several years. That monthly history is what the large tables contain.

The heavy computation runs **once**, to learn how accounts behave. What is learned is then applied to today's 7,211 delinquent customers. Large input, small output — which is normal, and is why the architecture in section 4.4 separates the two.

1.6 Why these questions are hard

- **The evidence is buried in scale.** Behavioural history lives in tables of 10 to 14 million rows. The answers are tables of a few dozen rows. Getting from one to the other is the entire task.
- **The obvious metric is misleading.** "Days past due" is easy to sort by, but it measures something different from "money we might still recover".
- **Nobody sees the counterfactual.** A badly ordered queue still looks busy. The loss is invisible unless someone measures it.

2. The data

The project uses **Home Credit Default Risk**, a real anonymised consumer-lending dataset published by Home Credit Group.

Table	Rows	One row represents
application	356,255	one loan application by one customer
bureau	1,716,428	one credit held at another institution
previous_application	1,670,214	one prior contract with this lender
pos_cash_balance	10,001,358	one month of a POS / cash loan account
credit_card_balance	3,840,312	one month of a credit-card account
installments_payments	13,605,401	one payment event (see section 4.1)
bureau_balance	27,299,925	one month of an external credit
Total	58,441,594	356,255 borrowers, 1,040,632 accounts

2.1 Three properties that shaped every later decision

Property	Consequence for the analysis
Four levels of grain person → contract → month → payment	A careless join multiplies rows. Measured later in the project: joining customers to payment events inflates a total by 37.5 times .
No calendar dates every date is a relative day-offset from each customer's own application	No calendar trends and no quarterly cohorts are possible. All time analysis had to be done in months-on-book instead.
No geography only an anonymised region rating of 1 to 3	Regional analysis is impossible. Recorded as a limitation rather than approximated with something weaker.

3. The approach

The work ran as a sequence of phases, each gated on evidence before the next began. One rule governed the whole project, and it overruled the plan three separate times.

Measure before deciding, and let the measurement overrule the assumption

What was assumed	What measurement showed
The payment table holds one row per installment	Wrong. One row is a payment <i>event</i> . 653,483 installments were settled by two or more payments; one by twelve.
Delinquency buckets should use the standard 30-day ladder	Only 542 accounts in the whole book ever peak in the 61-90 bucket. Too thin for stable rates, so two bucket schemes were built for two different purposes.
"Days late" should be measured at full settlement	Wrong. Lateness measured at the <i>first</i> payment separates the outcome far better (1.88x lift versus 1.33x).

Each of those was a reasonable prior belief that the data contradicted. Recording them, rather than silently correcting them, is part of what the project demonstrates.

3.1 The sequence of work

Stage	What it produced
Dataset selection	A documented search across 14 source families in three rounds, and an explicit justified choice
Structure inspection	Keys, grain, referential integrity and fan-out factors, all measured rather than assumed
Schema and loading	58.4M rows into MySQL, reconciled against source counts row for row
Data quality	15 checks run; 17 issues logged with detection method, treatment and business impact
Analytical layer	A pre-computed mart so downstream analysis runs on under 1M rows instead of 13.8M
Analysis	Portfolio, repayment behaviour, delinquency, roll rates, vintage curves, early-warning signals, collections priority
Validation and tuning	Every headline number re-derived three independent ways; query optimisation with execution plans

4. The work, step by step

4.1 Understanding the structure — and finding a documented assumption to be wrong

Before any analysis, each table was tested for key uniqueness. Six of the seven behaved as expected. The seventh, and the largest, did not.

The documentation implied `installments_payments` held one row per installment. Testing the key showed 653,483 duplicate rows. Investigation showed they were not errors at all:

```
SK_ID_PREV 2270983, installment 30, due day -2022, amount due 9,000
row 1: paid day -2046, amount 1,800 (24 days EARLY)
row 2: paid day -2019, amount 7,200 ( 3 days LATE)
total: 9,000 = exactly the amount due
```

One installment, settled by two separate payments. Across the table, **99.94%** of these multi-row cases summed exactly to the amount due. The real grain is **one row per payment event**.

Why this mattered more than it looks

Comparing `AMT_PAYMENT` to `AMT_INSTALLMENT` row by row, the natural thing to write, flags **1,295,493 rows (9.52%)** as underpayments. Aggregating first shows almost all of them were paid in full. That single mistake would have corrupted the entire repayment-behaviour analysis, and the resulting numbers would have looked completely plausible.

4.2 Loading 58.4 million rows

Three configuration problems were found before loading, and two file-format traps during it.

Issue	Resolution
<code>local_infile</code> disabled; <code>secure-file-priv</code> restricted the server to one folder	Used client-side <code>LOAD DATA LOCAL INFILE</code>
<code>innodb_buffer_pool_size</code> at the 128 MB installer default	Raised to 2 GB online, no restart
Quoted fields containing commas, e.g. Spouse, partner	<code>OPTIONALLY ENCLOSED BY ;</code> verified 11,370 such rows survived intact
Empty CSV fields become <code>0</code>, not <code>NULL</code> , in numeric columns	Every nullable column routed through <code>NULLIF(@var, '')</code> . Without this, 1,226,791 loans would have had a zero annuity and every exposure average would have been silently wrong.
Secondary indexes maintained across 13.6M inserts	Created after loading instead. All six built in 1.9 minutes.

Result: 58,441,594 rows loaded in 15.6 minutes, 0 rows skipped, every table count matching source exactly.

4.3 Data quality

Fifteen checks were run. Two examples show the kind of issue found.

A sentinel value that inverts a conclusion

18.01% of applications carried `DAYS_EMPLOYED = 365243`, which decodes to employment starting roughly 1,000 years in the future. Cross-tabulation identified it precisely:

Income type	Rows with sentinel	Share of group
Pensioner	55,352	99.98%
Unemployed	22	100.00%
All other income types	0	0.00%

Not corrupt data, but an encoding meaning "not currently employed". Left untreated it would feed a positive 1,000-year tenure into any analysis for a group that actually defaults *less* than average (5.40% against 8.66%), inverting the

conclusion.

Missing data that is itself informative

External credit scores were missing for a large share of customers. Testing the missingness against the outcome showed it was not random:

Group	Customers	Default rate
External score 3 present	246,546	7.77%
External score 3 missing	60,965	9.31%

Mean-filling those values would have destroyed a real signal and biased the sample at the same time. Missingness was kept as a flag instead.

4.4 Building an analytical layer

Analysis on the raw 13.8M-row monthly panel was measured at 275.7 seconds for a single roll-rate pass. Execution plans showed this could not be optimised away (see section 5.6), so the architecture changed instead: expensive computations run **once** into a pre-computed mart, and all analysis then runs against tables under 1M rows.

Mart table	Rows	Purpose
<code>mart_customer</code>	356,255	customer attributes plus behavioural aggregates and the outcome
<code>mart_account</code>	1,040,632	each account's current state plus its full history summarised
<code>mart_roll_rate</code>	40	the delinquency transition matrix
<code>mart_queue_tiered</code>	7,211	the collections queue, small enough to inspect by hand

Measured effect: the roll-rate computation went from **275.7 s to 0.09 s**, and a customer-level aggregation from **115.5 s to 4.26 s**.

4.5 What the analysis found

Finding	Evidence
The most delinquent accounts hold no money	Accounts 360+ days late: 1,369 accounts, mean balance 179 . Accounts 1-30 days late: 5,163 accounts holding 465.7M .
Ranking by severity reaches nothing	Working the top 10% of the queue by days-past-due reaches 0.0% of recoverable value. By money-at-risk times recovery-likelihood: 79.8% .
Recovery stops at 90 days	Accounts 1-30 days late recover on their own 55-59% of the time. Past 90 days that falls to 4-7% , and 85-98% simply stay put.
Utilisation warns before a payment is missed	Cards that have run over their limit are 27 times more likely to have been delinquent than cards kept under 30% utilisation.
Demographics do not explain who is behind	Delinquency is flat at 2.03% to 2.36% across every income type and age band tested.

4.6 The answer to the second question

Recall the question from section 1.4: when a customer with an existing loan applies for a new one, does their payment history on the old loan predict whether the new one will go bad?

Step one: does it predict at all?

Customers were grouped by how often they paid late, and each group's real outcome was measured against the overall base rate of **8.07%**.

Past repayment behaviour	Customers	Went bad on the new loan
Never paid late	170,110	7.28%
Late on up to 5% of installments	50,398	7.70%
Late on 5 to 15%	50,612	10.07%
Late on 15 to 30%	15,919	11.72%
Late on more than 30%	4,596	14.25%

The gradient is clean and it runs in the right direction — roughly double, worst group against best. So the answer to "does it predict" is **yes**.

The five warning signals

Each signal is a plain yes-or-no question about the customer, worth one point. Each was measured individually against the 8.07% base rate before any of them were combined.

#	Signal	What is actually tested	Fires on	Went bad	Lift
1	Ever 30+ days late	Any installment where the <i>first</i> payment arrived 30 or more days after the due date	4,558	15.18%	1.88x
2	Paid short	Paid less than the full amount due on at least one installment	3,089	15.83%	1.96x
3	Overdue elsewhere	Credit bureau shows a loan at <i>another lender</i> currently overdue	3,397	15.90%	1.97x
4	Getting worse	Days-late in the last three months is higher than in the three months before	30,373	10.38%	1.29x
5	Chronically behind	Has been in a delinquent state for three or more months over the account's life	30,820	9.13%	1.13x

The tension is visible immediately. Signals 1 to 3 are sharp, at roughly 1.9x the base rate, but each fires on only about 1% of customers. Signals 4 and 5 cover around 10% of the book but are weak. No signal is both sharp and broad, and that tension never resolves.

Step two: is it useful?

Predicting and being *operationally useful* are different things. The five signals were combined into a simple count — every customer scores 0 to 5 — and the real outcome measured in each group.

Signals firing	Customers	Actually went bad
0	242,267	7.76%
1	30,132	10.02%
2	16,273	10.03%
3	2,691	13.97%
4	270	20.00%

More signals means higher risk, so the framework does carry information. But scores of 1 and 2 are indistinguishable (10.02% against 10.03%), which is a genuine defect: the second signal a customer trips is usually one of the weak, high-volume ones and adds almost nothing. The operational trade-off is worse still.

Rule	Customers flagged	Share of the book	Of those, went bad	Share of all bad loans caught
1 or more signals	49,368	16.1%	10.29%	20.5%
3 or more signals	2,963	1.0%	14.55%	1.7%

Flag 16% of all customers and you catch one fifth of the bad loans — barely better than selecting at random. Tighten the rule until it is genuinely sharp and you are flagging 1% of the book and catching 1.7% of the problems. There is no threshold at which this framework is both accurate and broad.

Step three: the comparison that settled it

The five-signal framework was tested against the simplest alternative available — just looking at the customer's age.

Customer	Went bad on the new loan
Under 30, with zero warning signals	11.20%
Over 60, with two or more warning signals	7.90%

A young customer with a spotless payment record is riskier than an older customer with a bad one. **Age alone separates the outcome better than the entire behavioural framework does.**

What that result actually means — and what it does not

It would be easy to read the box above as "the signals do not work". That is the wrong conclusion. There are **two separate effects here, and both are real.**

Age band	Zero signals	Two or more signals
Under 30	11.20%	16.35%
30 to 39	9.22%	12.51%
40 to 49	7.32%	9.83%
50 to 59	5.83%	8.72%
60 and over	4.55%	7.90%

Read the table across, then down.

- **Across each row:** the signals work. In *every single age band*, risk rises by roughly 40 to 50% as signals accumulate. The framework is not broken.
- **Down the first column:** 11.20% falls to 4.55%. The age effect is a 2.5x spread, while the signal effect within a band is about 1.5x.

So when the two extremes are compared — young and clean against old and troubled — age wins, and the younger customer comes out riskier. Not because the signals are meaningless, but because **age moves the number further.**

The same thing, in a setting everyone knows

Car insurance. A 20-year-old with a perfect driving record against a 60-year-old with two speeding tickets: the 20-year-old is the bigger risk, because age dominates. That does not make the tickets meaningless — among 20-year-olds, the one with tickets is still riskier than the one without. Both facts are true at once, and a good pricing model uses both.

The honest conclusion

Past payment behaviour does predict future default, but weakly, and less well than a single demographic attribute. The recommendation reflects that: use the signals **on top of** other information rather than instead of it — as a **referral trigger**, a reason to examine an application more carefully, and never as a standalone rule to decline someone.

The reason the framework is weak is **coverage**, not signal quality. The sharpest individual signals fire on roughly 1% of customers each. Most applicants trip no signal at all, so the framework has nothing to say about them.

One practical point that reverses the ranking

Age is a **protected characteristic** under lending regulation in most markets, and generally cannot be used to decide a credit application. So although age is statistically the stronger predictor here, it is largely unusable. Behavioural signals matter *operationally* precisely because they are information a lender is permitted to act on. A statistic being stronger does not make it the one you can use.

Why a weak result is reported rather than quietly dropped

The easy version of this analysis stops at the first table, reports "past lateness nearly doubles default risk", and moves on. Testing it against the dumbest available alternative — and finding that the alternative wins — is what distinguishes a finding that can be defended from one that merely sounds good. The same discipline produced two other negative results in this project: a widely assumed risk indicator ("repeated credit enquiries") turned out to carry **no signal at all** at 1.04x the base rate, and one of the performance optimisations attempted failed outright and is documented in section 5.6.

5. The SQL

MySQL was the analytical engine, not a storage layer. Roughly **2,600 lines across 17 files**. The techniques below were used where a problem required them, not to demonstrate range.

Technique	The problem it solved here
Window functions (LAG)	Comparing each account-month to the previous one to build the transition matrix
ROW_NUMBER , NTILE	Ranking the same rows two different ways to compare orderings; exposure deciles
CTEs	Multi-step derivations kept readable and each step independently checkable
Conditional aggregation	Counting outcomes by category in one scan rather than several
Correlated and scalar subqueries	Comparing a computed figure against a stored one in the same result row
LEFT JOIN and orphan handling	Preserving the 4% of accounts whose parent contract is genuinely missing
HAVING	Suppressing groups too small to interpret, with counts always shown
EXPLAIN FORMAT=TREE	Diagnosing why one optimisation could not work

5.1 Finding the true grain CTE, aggregation

This is the query that overturned the documented assumption. It aggregates to the claimed key and asks whether the parts sum to the whole.

```
WITH per_installment AS (
  SELECT SK_ID_PREV, NUM_INSTALLMENT_VERSION, NUM_INSTALLMENT_NUMBER,
         COUNT(*)           AS n_payments,
         MAX(AMT_INSTALLMENT) AS amt_due,
         SUM(AMT_PAYMENT)    AS amt_paid
  FROM installments_payments
  GROUP BY SK_ID_PREV, NUM_INSTALLMENT_VERSION, NUM_INSTALLMENT_NUMBER
)
SELECT COUNT(*)           AS installments_total,
       SUM(n_payments > 1) AS installments_split,
       MAX(n_payments)    AS max_payments_one_installment,
       SUM(n_payments > 1 AND ABS(amt_paid - amt_due) < 0.01) AS split_and_fully_settled
FROM per_installment;
```

Result: 12,951,918 installments, 640,905 settled by multiple payments, a maximum of 12 payments for a single installment, and 99.94% summing exactly to the amount due. The `SUM(condition)` form works because MySQL evaluates a boolean to 1 or 0, so a count and a filter happen in one pass.

5.2 The roll-rate matrix LAG, window partitioning, gap guard

The core delinquency question: what is the probability an account in one state this month is in another state next month? This requires comparing consecutive rows within each account.

```
WITH seq AS (
  SELECT PRODUCT, SK_ID_PREV, MONTHS_BALANCE, BUCKET_ANALYTICAL,
         LAG(BUCKET_ANALYTICAL) OVER (PARTITION BY SK_ID_PREV, PRODUCT
                                     ORDER BY MONTHS_BALANCE) AS from_bucket,
         LAG(MONTHS_BALANCE) OVER (PARTITION BY SK_ID_PREV, PRODUCT
                                   ORDER BY MONTHS_BALANCE) AS prev_mb
  FROM stg_account_month
)
SELECT PRODUCT, from_bucket, BUCKET_ANALYTICAL AS to_bucket, COUNT(*) AS transitions
FROM seq
WHERE from_bucket IS NOT NULL
  AND MONTHS_BALANCE = prev_mb + 1 -- consecutive months only
GROUP BY PRODUCT, from_bucket, BUCKET_ANALYTICAL;
```

The line that matters most

`AND MONTHS_BALANCE = prev_mb + 1`. Consecutive *rows* are not necessarily consecutive *months* — the panel has gaps. Without this guard, a jump from month -8 to month -3 would be recorded as a single month's transition, inventing movements that never happened. The number of pairs rejected for this reason was measured and reported (0.02%) rather than assumed to be zero.

5.3 Comparing two orderings of the same rows dual ROW_NUMBER, CROSS JOIN

The project's central result. The same 7,211 accounts are ranked two ways, and the question asked is how much recoverable value each ordering reaches for a given amount of effort.

```
WITH ranked AS (
  SELECT cure_weighted_exposure AS v,
         ROW_NUMBER() OVER (ORDER BY cure_weighted_exposure DESC) AS rn_value,
         ROW_NUMBER() OVER (ORDER BY DPD_NOW DESC) AS rn_dpd,
         COUNT(*) OVER () AS n,
         SUM(cure_weighted_exposure) OVER () AS total_value
  FROM mart_collections_queue
)
SELECT pct_worked,
       ROUND(100.0*SUM(CASE WHEN rn_value <= n*pct_worked/100 THEN v ELSE 0 END)
            /MAX(total_value), 2) AS pct_value_reached_by_VALUE,
       ROUND(100.0*SUM(CASE WHEN rn_dpd <= n*pct_worked/100 THEN v ELSE 0 END)
            /MAX(total_value), 2) AS pct_value_reached_by_DPD
FROM ranked
CROSS JOIN (SELECT 5 AS pct_worked UNION SELECT 10 UNION SELECT 20
            UNION SELECT 30 UNION SELECT 50) p
GROUP BY pct_worked ORDER BY pct_worked;
```

Two `ROW_NUMBER` windows over the same rows give two orderings side by side. The `CROSS JOIN` against a small inline list produces every capacity level in one query instead of five. The empty `OVER ()` frames compute whole-set totals without a second pass.

Capacity worked	Ranked by value	Ranked by DPD
5%	57.59%	0.00%
10%	79.82%	0.00%
20%	96.29%	0.00%

5.4 Evaluating signals against an observed outcome CTE, CROSS JOIN, conditional aggregation

Rather than asserting that certain behaviours indicate risk, each candidate signal was measured against the observed 8.0729% base rate, and the precision-versus-coverage trade-off computed at every threshold.

```
WITH scored AS (
  SELECT TARGET,
    (N_LATE_FIRST_30PLUS > 0) + (N_SHORTFALL > 0) + (N_BUREAU_OVERDUE > 0)
    + (N_DETERIORATING_ACCOUNTS > 0) + (MONTHS_DELINQUENT >= 3) AS sc
  FROM mart_customer WHERE SOURCE = 'train'
), tot AS (SELECT COUNT(*) n, SUM(TARGET) d FROM scored)
SELECT th AS threshold,
  SUM(sc >= th) AS flagged,
  ROUND(100.0*SUM(CASE WHEN sc >= th THEN TARGET ELSE 0 END)
    / NULLIF(SUM(sc >= th),0), 3) AS precision_pct,
  ROUND(100.0*SUM(CASE WHEN sc >= th THEN TARGET ELSE 0 END)
    / MAX(tot.d), 3) AS recall_pct
FROM scored CROSS JOIN tot
CROSS JOIN (SELECT 1 AS th UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) thr
GROUP BY th ORDER BY th;
```

Adding booleans gives a signal count without a `CASE` ladder. The result was reported honestly: the framework reaches only a 1.80x lift at usable volume, and scores of 1 and 2 turned out statistically indistinguishable — a genuine weakness that was published rather than hidden.

5.5 Demonstrating the fan-out error join cardinality

The same business question asked two ways, to quantify what a careless join costs.

```
-- CORRECT: the customer table is already at customer grain
SELECT NAME_INCOME_TYPE, SUM(AMT_CREDIT) FROM mart_customer GROUP BY NAME_INCOME_TYPE;

-- WRONG: joining to payment events repeats AMT_CREDIT once per payment
SELECT c.NAME_INCOME_TYPE, SUM(c.AMT_CREDIT)
FROM mart_customer c JOIN stg_installment i ON i.SK_ID_CURR = c.SK_ID_CURR
GROUP BY c.NAME_INCOME_TYPE;
```

Method	Result	Rows aggregated	Runtime
Correct	209.40 bn	356,255	0.69 s
Fanned out	7,843.74 bn	12,951,918	96.42 s

The wrong answer is **37.5 times too large** and 140 times slower. The detail that proves the point: rows inflated 36.4x but the value inflated 37.5x. If the multiplier were constant it could be divided out; because it varies per customer, the join silently re-weights the whole portfolio toward customers with more payment history.

5.6 An optimisation that failed, and why that is worth reporting

The roll-rate query took 275.7 seconds. The hypothesis was that it partitioned by `SK_ID_PREV` while the primary key was `(SK_ID_PREV, PRODUCT, MONTHS_BALANCE)`, forcing an unnecessary sort. Aligning them should have let the engine read rows already in order.

```
-> Window aggregate with buffering: lag(...) OVER (PARTITION BY SK_ID_PREV ORDER BY MONTHS_BALANCE)
-> Sort: SK_ID_PREV, MONTHS_BALANCE (cost=1.45e+6 rows=13.2e+6)
-> Index scan using ix_am_bucket
```

The hypothesis was refuted: 253.4 s against 271.9 s, and both plans performed a full sort of 13.2 million rows regardless. MySQL 8 always materialises and sorts for a buffered window function and will not use clustered-index order to avoid it. This is an engine limitation, not a query-writing mistake, and no index fixes it.

The response was to stop optimising the query and change the architecture: compute the matrix once into a 40-row table. **275.7 s became 0.09 s.**

Two optimisations that did work

Change	Before	After	Why
Covering index on the live book	15.27 s	0.85 s	The index holds every column the query needs, so the table is never touched
Materialising the transition matrix	275.70 s	0.09 s	An unavoidable sort is paid once instead of on every run

6. Python

Python was used deliberately as a *secondary* tool, for the things SQL is not suited to, rather than duplicating work already done in SQL.

6.1 Independent validation

Everything from the analytical layer onwards read from the mart. A single error in building that mart would have propagated silently through every result that followed. So every headline number was re-derived by a third, fully separate path: **pandas reading the original CSV files**, never touching MySQL.

```
ip = pd.read_csv("installments_payments.csv",
                 usecols=["SK_ID_PREV", "NUM_INSTALLMENT_VERSION", "NUM_INSTALLMENT_NUMBER"])
check("split payments (F1)", len(ip) - len(ip.drop_duplicates()))

def latest(df):
    """One row per account: its highest (latest) MONTHS_BALANCE."""
    idx = df.groupby("SK_ID_PREV")["MONTHS_BALANCE"].idxmax()
    return df.loc[idx]

pos_1, card_1 = latest(pos), latest(card)
check("accounts at latest month", len(pos_1) + len(card_1))
```

Three paths therefore produce every key figure: the analytical pipeline, a separate SQL query against the base tables, and pandas against the source CSVs. Different engines, different languages, different code.

Figure	Pipeline	Independent SQL	pandas
Accounts at latest month	1,040,632	1,040,632	1,040,632
Active accounts	327,101	327,101	327,101
Delinquent accounts	7,211	7,211	7,211
Card exposure	7,774.0M	7,774.0M	7,774.0M
Default base rate	8.0729%	8.0729%	8.0729%
Split payments	653,483	653,483	653,483

20 reconciliation checks in total. All exact. Three independent derivations agreeing is a far stronger claim than any single computation, and it is what allows these numbers to be stated without hedging.

6.2 Profiling 58 million rows within 7.3 GB of memory

The initial data profiling ran on the raw CSVs before anything reached MySQL, on a machine with 7.3 GB of RAM against 2.68 GB of files. This required chunked, streaming accumulation rather than loading tables whole:

```
for ch in pd.read_csv(path, chunksize=1_500_000, low_memory=False):
    for c in ch.columns:
        nulls[c] += int(ch[c].isna().sum())
        if pd.api.types.is_numeric_dtype(ch[c]):
            v = ch[c].dropna()
            mn[c] = v.min() if c not in mn else min(mn[c], v.min())
            mx[c] = v.max() if c not in mx else max(mx[c], v.max())
    del ch; gc.collect()
```

The same approach checked key uniqueness on a 27.3M-row table by loading only the two key columns, keeping memory bounded regardless of table width.

6.3 Code generation

The load scripts were generated rather than hand-written. With 219 columns across eight files, every nullable column needed `NULLIF(@var, '')` handling and every column list had to match its CSV header exactly. A Python

script parsed the schema for nullability, read each CSV header, and emitted the `LOAD DATA` statements — removing an entire class of transcription error that would have been invisible until the numbers came out wrong.

7. Running it yourself: the queries and their outputs

Everything below can be pasted straight into MySQL Workbench after `USE loan_portfolio;`. All of them are read-only. They are grouped by the two questions from section 1, and each shows what it returns so the output can be checked against the expected result.

7.1 Question one — the collections manager's queries

Q1. How big is my problem this morning?

```
SELECT PRODUCT,
       COUNT(*) AS accounts,
       SUM(IS_ACTIVE) AS active,
       ROUND(SUM(CASE WHEN IS_ACTIVE=1 THEN EXPOSURE END)/1000000,1) AS exposure_m,
       SUM(IS_ACTIVE=1 AND DPD_NOW>0) AS delinquent
FROM mart_account
GROUP BY PRODUCT WITH ROLLUP;
```

PRODUCT	accounts	active	exposure_m	delinquent
CARD	104,307	90,952	7,774.0	1,317
POS	936,325	236,149	44,444.3	5,894
(total)	1,040,632	327,101	52,218.2	7,211

Reads as: 327,101 live accounts, 7,211 of them behind. That 7,211 is the queue.

Q2. Where is the money? (the query that changes the decision)

```
SELECT BUCKET_ANALYTICAL AS bucket,
       COUNT(*) AS accounts,
       ROUND(SUM(EXPOSURE)/1000000, 2) AS exposure_m,
       ROUND(AVG(EXPOSURE), 0) AS mean_balance,
       ROUND(AVG(cure_rate)*100, 2) AS recovers_pct,
       ROUND(SUM(cure_weighted_exposure)/1000000, 2) AS recoverable_m
FROM mart_collections_queue
GROUP BY BUCKET_ANALYTICAL
ORDER BY FIELD(BUCKET_ANALYTICAL, '1-30', '31-90', '91-360', '360+');
```

bucket	accounts	exposure_m	mean_balance	recovers_pct	recoverable_m
1-30	5,163	465.66	91,648	55.40	259.61
31-90	372	101.21	281,140	24.52	23.97
91-360	307	18.21	63,226	5.99	1.21
360+	1,369	0.20	179	2.04	0.00

Reads as: the most overdue group has 1,369 customers owing an average of 179 each, and recovers 2% of the time. The mildest group has 5,163 customers owing 465.7 million. This single table is the reason the queue is not sorted by severity.

Q3. Prove the new ordering is better than the obvious one

```

WITH ranked AS (
  SELECT cure_weighted_exposure AS v,
         ROW_NUMBER() OVER (ORDER BY cure_weighted_exposure DESC) AS rn_value,
         ROW_NUMBER() OVER (ORDER BY DPD_NOW DESC) AS rn_dpd,
         COUNT(*) OVER () AS n,
         SUM(cure_weighted_exposure) OVER () AS total_value
  FROM mart_collections_queue
)
SELECT pct_worked AS pct_of_capacity,
       ROUND(100.0*SUM(CASE WHEN rn_value <= n*pct_worked/100 THEN v ELSE 0 END)
            /MAX(total_value), 2) AS reached_sorting_by_VALUE,
       ROUND(100.0*SUM(CASE WHEN rn_dpd <= n*pct_worked/100 THEN v ELSE 0 END)
            /MAX(total_value), 2) AS reached_sorting_by_DPD
FROM ranked
CROSS JOIN (SELECT 5 AS pct_worked UNION SELECT 10 UNION SELECT 20
            UNION SELECT 30 UNION SELECT 50) p
GROUP BY pct_worked ORDER BY pct_worked;

```

pct_of_capacity	sorting by VALUE	sorting by DPD
5	57.59%	0.00%
10	79.82%	0.00%
20	96.29%	0.00%
50	100.00%	27.89%

Reads as: spend a day working the top 10% either way. One ordering reaches 79.8% of the recoverable money; the other reaches none of it.

Q4. Give me the list to work today

```

SELECT SK_ID_PREV AS account_id,
       PRODUCT, BUCKET_ANALYTICAL AS bucket, DPD_NOW AS days_late,
       ROUND(EXPOSURE, 0) AS balance,
       ROUND(cure_rate*100, 1) AS recovers_pct,
       ROUND(cure_weighted_exposure,0) AS recoverable,
       priority_tier
FROM mart_queue_tiered
ORDER BY cure_weighted_exposure DESC
LIMIT 25;

```

Returns: 25 rows, topped by POS account 1735470 — only **1 day** late, but a balance of 2,202,336 and a 55% recovery rate, giving 1,211,505 of recoverable value. Sorting by days-late, that customer would never have been called.

7.2 Question two — the credit decision queries

Q5. Does past payment behaviour predict anything?

```
SELECT CASE WHEN PCT_LATE_FIRST IS NULL THEN '6: no history'
           WHEN PCT_LATE_FIRST = 0 THEN '1: never late'
           WHEN PCT_LATE_FIRST <= 5 THEN '2: late up to 5%'
           WHEN PCT_LATE_FIRST <= 15 THEN '3: late 5-15%'
           WHEN PCT_LATE_FIRST <= 30 THEN '4: late 15-30%'
           ELSE '5: late over 30%' END AS repayment_history,
COUNT(*) AS customers,
SUM(TARGET) AS went_bad,
ROUND(100.0*AVG(TARGET), 2) AS went_bad_pct,
ROUND(AVG(TARGET)/0.080729, 2) AS vs_base_rate
FROM mart_customer WHERE SOURCE='train'
GROUP BY repayment_history ORDER BY repayment_history;
```

repayment_history	customers	went_bad_pct	vs_base_rate
1: never late	170,110	7.28	0.90
3: late 5-15%	50,612	10.07	1.25
5: late over 30%	4,596	14.25	1.77

Reads as: yes. Worst group is roughly double the best. `SOURCE='train'` restricts to the 307,511 customers whose real outcome is known.

Q6. Do the five signals combined work?

```
SELECT signal_count,
COUNT(*) AS customers,
ROUND(100.0*AVG(TARGET), 2) AS went_bad_pct,
ROUND(AVG(TARGET)/0.080729, 2) AS vs_base_rate
FROM (
  SELECT TARGET,
    (COALESCE(N_LATE_FIRST_30PLUS,0) > 0) -- 1 ever 30+ days late
    + (COALESCE(N_SHORTFALL,0) > 0) -- 2 paid short
    + (COALESCE(N_BUREAU_OVERDUE,0) > 0) -- 3 overdue elsewhere
    + (COALESCE(N_DETERIORATING_ACCOUNTS,0) > 0) -- 4 getting worse
    + (COALESCE(MONTHS_DELINQUENT,0) >= 3) -- 5 chronically behind
    AS signal_count
  FROM mart_customer WHERE SOURCE='train'
) t
GROUP BY signal_count ORDER BY signal_count;
```

Returns: 0 signals = 7.76%, 1 = 10.02%, 2 = 10.03%, 3 = 13.97%, 4 = 20.00%. Rising, but note scores 1 and 2 are effectively identical. Adding booleans works because MySQL evaluates each comparison to 1 or 0.

Q7. The decisive test — do the signals beat simply looking at age?

```
SELECT CASE WHEN AGE_YEARS < 30 THEN '1: under 30' WHEN AGE_YEARS < 40 THEN '2: 30-39'
        WHEN AGE_YEARS < 50 THEN '3: 40-49'    WHEN AGE_YEARS < 60 THEN '4: 50-59'
        ELSE '5: 60+' END                      AS age_band,
COUNT(*)                                     AS customers,
ROUND(100.0*AVG(CASE WHEN sc = 0 THEN TARGET END), 2) AS zero_signals_pct,
ROUND(100.0*AVG(CASE WHEN sc >= 2 THEN TARGET END), 2) AS two_plus_signals_pct
FROM (
    SELECT AGE_YEARS, TARGET,
        (COALESCE(N_LATE_FIRST_30PLUS,0) > 0) + (COALESCE(N_SHORTFALL,0) > 0)
        + (COALESCE(N_BUREAU_OVERDUE,0) > 0) + (COALESCE(N_DETERIORATING_ACCOUNTS,0) > 0)
        + (COALESCE(MONTHS_DELINQUENT,0) >= 3) AS sc
    FROM mart_customer WHERE SOURCE='train'
) t
GROUP BY age_band ORDER BY age_band;
```

age_band	customers	zero_signals_pct	two_plus_signals_pct
1: under 30	44,738	11.20	16.35
3: 40-49	76,602	7.32	9.83
5: 60+	35,665	4.55	7.90

Reads as: across every row the signals work. But down the first column the age effect is larger, so 11.20% (young, clean) exceeds 7.90% (old, two or more signals). This is the query that produced the conclusion in section 4.6.

8. Tools

Tool	Role in the project
MySQL 8.0	The analytical engine. Schema design, bulk loading, all analysis, query optimisation. Roughly 2,600 lines across 17 files.
Python / pandas	Memory-bounded profiling of 58M rows, independent validation against source files, and generation of the load scripts.
MySQL Workbench	Interactive query development and inspection of results.
PowerShell	Pipeline automation, timing instrumentation, credential handling via MySQL's own credential store so no password appears in any script.

9. Outcome

Deliverable	Detail
A ranked collections queue	7,211 accounts, ordered by money at risk times observed recovery likelihood. Top 20% of the queue holds 96.3% of recoverable value.
A measured case for re-ordering	Working the top 10% by severity reaches 0.0% of recoverable value; by value it reaches 79.8%.
A delinquency transition matrix	Quantifies that recovery effectively stops at 90 days, from 12.5M observed month-to-month transitions.
A tested early-warning framework	Five transparent rules, validated against an observed 8.07% base rate — and reported as modest (1.80x lift) rather than oversold.
A full audit trail	17 data-quality issues, 11 assumptions with rejected alternatives, 17 insights rated by confidence, and 10 limitations, all documented.

What the project demonstrates

Working at scale (58.4M rows), diagnosing performance from execution plans rather than guesswork, discovering that a documented assumption about the data was wrong and quantifying what it would have cost, validating every number across three independent engines, and reporting negative results — a signal that did not work, a framework weaker than expected, an optimisation that failed — alongside the positive ones.

The final recommendation list includes one item that explicitly recommends *not* acting: a 6.5x difference between loan cohorts was found, and reported as a flag for investigation rather than a conclusion, because three confounding factors could not be separated with this data.